Google Cloud - Community

Bash to Terraform with Gemini 1.5 Flash

Overview

Yannipeng 11 min read · Aug 14, 2024

In today's cloud-driven world, managing infrastructure efficiently is critical for ensuring scalability, reliability, and maintainability. Bash scripts have long been a staple for automating tasks in Google Cloud Platform (GCP), but they often fall short in terms of readability, maintainability, and scalability as the complexity of infrastructure grows. Terraform, an infrastructure as code (IaC) tool, offers a powerful alternative by allowing infrastructure to be described in high-level configuration files that are easy to read, version, and reuse.

Why Use GenAI For Script Transformation

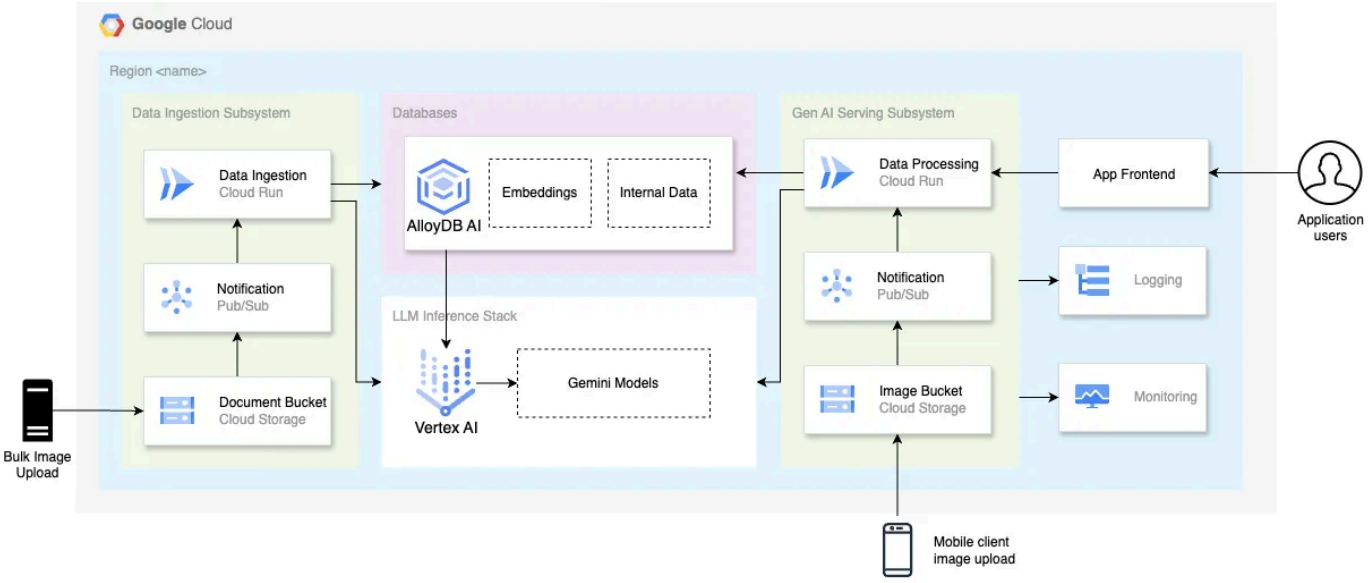
We take an innovative approach to automating the transformation of bash scripts into Terraform configurations using the Gemini 1.5 Flash Large Language Model (LLM) and RAG application. This transformation not only

simplifies the management of GCP resources but also ensures adherence to best practices in Terraform. The application we've developed leverages cutting-edge technologies, including Java, Spring, Spring AI (AI orchestration framework utilizing the Vertex AI SDK), Vertex AI, Vector Embeddings, Cloud Storage, Eventarc, and AlloyDB, to create a seamless and efficient process for infrastructure automation.

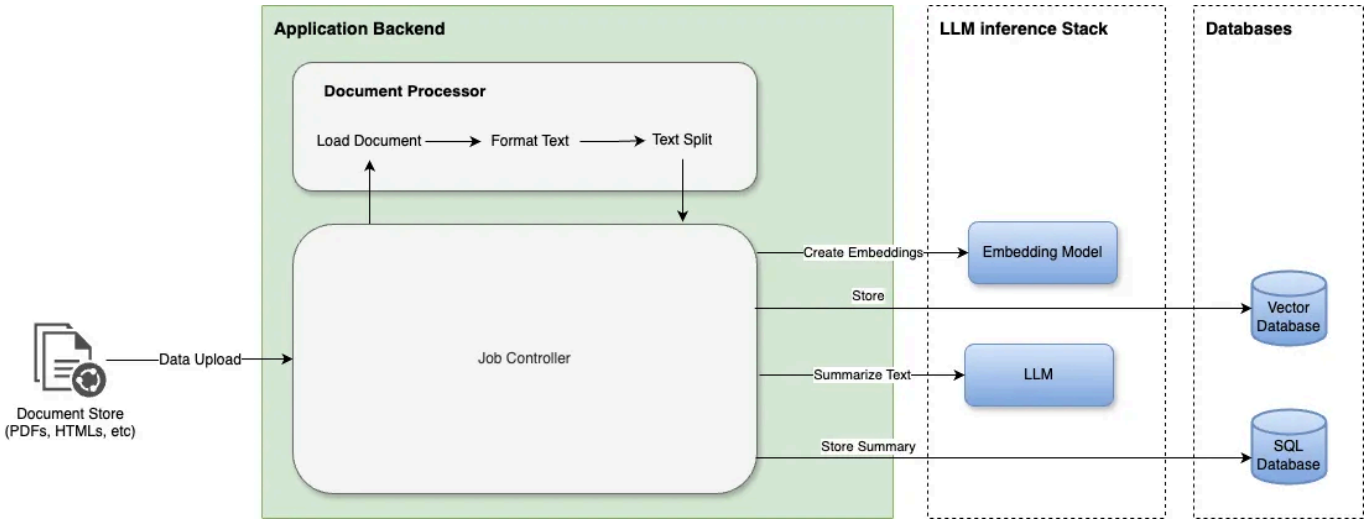
In the following sections, we'll go into the architecture and implementation of our solution, demonstrating how these technologies come together to transform bash scripts into well-structured Terraform code using RAG to optimize the output of the LLM.

Architecture:

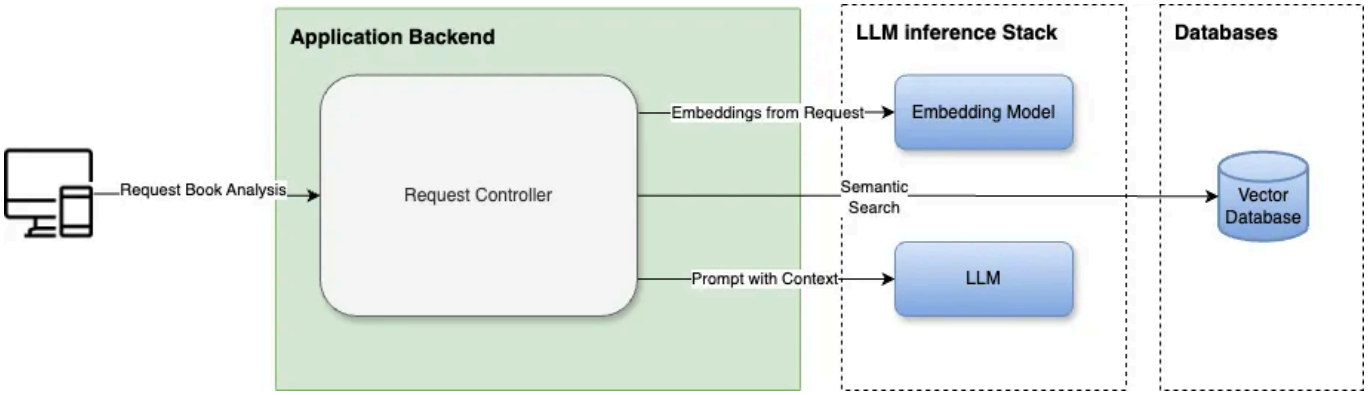
GCP architecture: GCP cloud services are involved in the RAG application.



Ingestion Flow: This involves collecting and processing Terraform best practices, converting them into embeddings, and storing them in the AlloyDB database.



Analysis Flow: It includes utilizing semantic search to retrieve relevant best practices, constructing a prompt for the Gemini 1.5 Flash LLM, and generating well-structured Terraform code from the provided bash script.



*More details in the section below

Implementing Ingestion & Analysis Architecture

Below are key code snippets for implementing the ingestion and analysis flows of our application. For complete source code check references.

Ingestion Flow:

The ingestion implementation is triggered by an upload to cloud storage, and the content of each chunk in the best practices Terraform book is inserted into the library database. Each chunk or page of content is automatically turned into vector embeddings using AlloyDB and Vertex AI integration. Since the model (textembedding-gecko@003) used for this transformation is defined directly in the database schema's Data Definition Language (DDL), the transformation happens seamlessly without requiring explicit handling in the source code. These embeddings represent the semantic meaning of the text, enabling efficient and accurate retrieval during the analysis phase. Once stored in AlloyDB, these best practices will later be utilized in the prompt of the transformation using RAG. This approach ensures that the embeddings are readily available and optimized for retrieval, streamlining the process and enhancing the efficiency of the entire workflow. Consequently, the integration between AlloyDB and Vertex AI simplifies the ingestion process, allowing for a robust and scalable solution.

```
public Integer insertPages(Integer bookId, String content, Integer pageNumber) {
    String sql = "insert into pages (\n" +
    "book_id,\n" +
    "content,\n" +
    " page_number)\n" +
    "values (?,?,?)";
    Object[] parameters = new Object[]{bookId, content, pageNumber};
    int success =jdbcTemplate.update(sql, parameters);
    return success;
}
```

*Note the following best practice documentations were ingested to alloy

Analysis Flow:

The analysis flow of our application is designed to transform bash scripts into well-structured Terraform configurations. This process uses Retrieval-Augmented Generation (RAG) to build a prompt that incorporates best practices for writing Terraform scripts. By retrieving the most relevant information from our AlloyDB database, called “library,” and combining it with the user’s input bash script, we ensure that the generated Terraform code is both accurate and adheres to best practices. The prompt template is linked here. The Reference Documentation used to generate the output will be retrieved using a semantic search using embeddings generated by textembedding-gecko in Vertexai.

Storing and retrieving embeddings in AlloyDB involves setting up the appropriate schema and implementing the database interaction methods.

Schema Definition:

```
# Authors Table *Note the embeddings llm is defined on the column embeddings
CREATE TABLE authors (
  author_id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  bio TEXT,
  embedding public.vector GENERATED ALWAYS AS (public.embedding('textembedding-g
);
 - Books Table
CREATE TABLE books (
  book_id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  author_id INT NOT NULL,
  publication_year DATE,
  scope scope_type NOT NULL DEFAULT 'public',
  CONSTRAINT fk_author
  FOREIGN KEY(author_id)
  REFERENCES authors(author_id)
);
# Pages Table *Note the embeddings llm is defined on the column embeddings
CREATE TABLE pages (
  page_id SERIAL PRIMARY KEY,
```

```
  book_id INT NOT NULL,
  page_number INT NOT NULL,
  content TEXT,
  embedding public.vector GENERATED ALWAYS AS (public.embedding('textembedding-g
  CONSTRAINT fk_pages
  FOREIGN KEY(book_id)
  REFERENCES books(book_id)
);
```

Read operation & Semantic Search:

```
import org.springframework.jdbc.core.JdbcTemplate;

public List<Map<String, Object>> promptForBooks(String prompt, String book, Stri
    if (characterLimit == null || characterLimit == 0) {
    characterLimit = 2000;
    }
    String sql = "SELECT\n" +
    " b.title,\n" +
    " left(p.content, ?) as page,\n" +
    " a.name,\n" +
    " p.page_number,\n" +
    " (p.embedding <=> embedding('textembedding-gecko@003', ?)::vector) as dista
    "FROM\n" +
    " pages p\n" +
    "JOIN books b on\n" +
    " p.book_id = b.book_id\n" +
    "JOIN authors a on\n" +
    " a.author_id = b.author_id\n";

    Object[] parameters = new Object[]{characterLimit, prompt, book, author};
    List<Object> params = Arrays.stream(parameters)
    .filter(Objects::nonNull)
    .filter(p -> p instanceof String ? !((String) p).isEmpty() : true)
    .collect(Collectors.toList());
    if (params.size() > 2) {
    sql += createWhereClause(book, author);
    }
    sql += " ORDER BY distance ASC LIMIT 10;";
```

```
        return jdbcTemplate.queryForList(sql, params.toArray());
    }
```

Prompting the Model:

Using the following code to prompt the model ensures that the transformation process is efficiently and robustly handled. The prompt template used and a detailed explanation of the prompt used in the following method here.

```
import org.springframework.ai.vertexai.gemini.VertexAiGeminiChatModel;
import org.springframework.ai.vertexai.gemini.VertexAiGeminiChatOptions;

public String promptModel(String prompt, String model) {
    long start = System.currentTimeMillis();
    logger.info("Chat model prompt: {} …", prompt.substring(0, Math.min(500, pro
    int maxRetries = 3;
    int retryCount = 0;
    ChatResponse chatResponse = null;
    while (retryCount < maxRetries) {
    try {
      chatResponse = chatClient.call(new Prompt(prompt,
        VertexAiGeminiChatOptions.builder()
        .withTemperature(0.4f)
        .withModel(model)
        .build())
      );
      if (chatResponse.getResult() != null) {
        break;
      } else {
        logger.warn("Received invalid response from model. Retrying…");
      }
    } catch (Exception exception) {
      logger.error("Error calling chat model: ", exception);
    }
      retryCount++;
    }
    logger.info("Elapsed time ( {}, with SpringAI): {} ms", model, (System.curre
    String output = VertexModels.RETRY_MSG;
```

```
        if (chatResponse != null && chatResponse.getResult() != null) {
          output = chatResponse.getResult().getOutput().getContent();
        }
        logger.info("Chat model output: {} …", output.substring(0, Math.min(1000, ou
        return output;
    }
```

Build:

Setting up Spring and Spring AI to interact with Vertex AI involves configuring the necessary dependencies and properties.

*Maven pom is referenced in the git repo here: pom.xml

By following these steps, you can implement the ingestion and analysis flows of your application, set up Spring and Spring AI for interacting with Vertex AI, and handle database interactions using AlloyDB effectively.

Setting up the Cloud Infra

Once local development of the RAG application is complete the next step is to provision the cloud infrastructure so it can begin serving requests. Many developers may begin this process by running a series of “gcloud cli” commands in a bash script, or manually. We developed a Genai application that harnesses the power of Retrieval-Augmented Generation (RAG) to streamline the conversion of shell scripts into Terraform configurations. This application addresses a common challenge faced by cloud engineers: transitioning from traditional, manual scripting to a more scalable and maintainable Infrastructure as Code (IaC) approach. By leveraging RAG, we can automate this process, making it faster and more efficient.

The following is a sample bash script that provisions a piece of the cloud infra for a gcp environment. After manually running through these commands it was time to streamline the cloud provisioning process by converting the shell script to terraform.

Bash

```
#!/bin/bash
# Script to transform to terraform
PROJECT_ID="my-gcp-project"
ZONE="us-central1-a"
INSTANCE_NAME="web-server-01"
MACHINE_TYPE="e2-medium"
IMAGE="debian-cloud/debian-11"

# Enable required services
gcloud services enable vision.googleapis.com
gcloud services enable cloudfunctions.googleapis.com

# Create a Compute Engine instance
gcloud compute instances create $INSTANCE_NAME \
  --project $PROJECT_ID \
  --zone $ZONE \
  --machine-type $MACHINE_TYPE \
  --image $IMAGE \
  --boot-disk-size 20GB

# Create a Google Cloud Storage bucket
gsutil mb -l us-central1 gs://library_next25_images
gsutil uniformbucketlevelaccess set on gs://library_next25_images
gsutil iam ch allUsers:objectViewer gs://library_next25_images
```

*Reference “**Prompting the Model:**” above for details on RAG, prompting, and source code.

Testing End-to-end

```
#!/bin/bash
# Script to transform to terraform
PROJECT_ID="my-gcp-project"
ZONE="us-central1-a"
INSTANCE_NAME="web-server-01"
MACHINE_TYPE="e2-medium"
IMAGE="debian-cloud/debian-11"

# Enable required services
gcloud services enable vision.googleapis.com
gcloud services enable cloudfunctions.googleapis.com

# Create a Compute Engine instance
gcloud compute instances create $INSTANCE_NAME \
  --project $PROJECT_ID \
  --zone $ZONE \
  --machine-type $MACHINE_TYPE \
  --image $IMAGE \
  --boot-disk-size 20GB

# Create a Google Cloud Storage bucket
gsutil mb -l us-central1 gs://library_next25_images
gsutil uniformbucketlevelaccess set on gs://library_next25_images
gsutil iam ch allUsers:objectViewer gs://library_next25_images
```

```
# Curl command to send the script above as a payload
curl --location 'https://hostname/document/bash/to-terraform' \
--header 'Content-Type: application/json' \
--data '{ "script": "$SCRIPT" }'
```

Output

```
## Terraform Conversion: Bash Script to Terraform

Here's the Terraform code, organized into separate files, along with explanation

**1. Terraform Code in Separate Files:**
```

variables.tf:

```terraform
variable "project_id" {
  type = string
  description = "GCP Project ID"
  default = "my-gcp-project"
}

variable "zone" {
  type = string
  description = "Compute Engine Zone"
  default = "us-central1-a"
}

variable "instance_name" {
  type = string
  description = "Name of the Compute Engine Instance"
  default = "web-server-01"
}

variable "machine_type" {
  type = string
  description = "Machine Type for the Instance"
  default= "e2-medium"
}

variable "image" {
  type = string
  description = "Image to use for the Instance"
  default= "debian-cloud/debian-11"
}

variable "boot_disk_size" {
  type = number
  description = "Size of the boot disk in GB"
  default = 20
}

variable "bucket_name" {
  type = string
  description = "Name of the Cloud Storage Bucket"
  default = "library_next25_images"
}

variable "location" {
  type = string
  description = "Location of the Cloud Storage Bucket"
  default = "us-central1"
}
```

```

**main.tf:**

```terraform
# Enable required Google Cloud services
resource "google_project_service" "vision" {
  service = "vision.googleapis.com"
  disable_on_destroy = false
}

resource "google_project_service" "cloudfunctions" {
  service = "cloudfunctions.googleapis.com"
  disable_on_destroy = false
}

# Create Compute Engine Instance
resource "google_compute_instance" "web_server" {
  name         = var.instance_name
  machine_type = var.machine_type
  zone         = var.zone
  project      = var.project_id

  boot_disk {
    initialize_params {
      image = var.image
      size = var.boot_disk_size
      type = "pd-standard"
    }
    auto_delete = true
  }
  network_interface {
    network = "default"
  }
}

# Create Cloud Storage Bucket
resource "google_storage_bucket" "images" {
  name     = var.bucket_name
  location = var.location
  force_destroy = true

  uniform_bucket_level_access = true
}

# Grant allUsers:objectViewer role to the bucket
resource "google_storage_bucket_iam_member" "allUsers_objectViewer" {
  bucket = google_storage_bucket.images.name
  role   = "roles/storage.objectViewer"
  member = "allUsers"
```
```

```
}
```

2. Explanation:

* **variables.tf:**
    * Defines all input variables used in `main.tf` file.
    * Provides descriptions for each variable, making the code more readable.
    * Sets default values for variables that are common or have a reasonable def
    * `bucket_name` and `location` are introduced for the bucket configuration.
* **main.tf:**
    * **Google Project Service (Vision & Cloud Functions):**
        * Uses the `google_project_service` resource to enable the required Goog
        * `disable_on_destroy` ensures that the services remain enabled even if
    * **Compute Engine Instance:**
        * Uses the `google_compute_instance` resource to create the VM instance
        * `boot_disk` configures the instance's boot disk using the provided ima
        * `network_interface` attaches the instance to the default network.
    * **Cloud Storage Bucket:**
        * Uses the `google_storage_bucket` resource to create the bucket with na
        * `force_destroy` ensures the bucket is deleted even if it contains obje
        * `uniform_bucket_level_access` enforces the bucket to have uniform acce
    * **IAM Binding (Bucket Access):**
        * Uses the `google_storage_bucket_iam_member` resource to grant `allUser

3. Best Practices:

* **File Organization:** The configuration is divided into separate files (`main
* **Resource Naming:** Resources are named using descriptive names, e.g., `web_s
* **Variable Usage:** All configuration parameters are defined as variables in `
* **Descriptions:** All resources and variables have descriptive comments to imp
* **Output Attributes:** The code references the `id` attribute of the `google_s

4. Potential Optimizations:

* **Module Usage:** If you're creating many similar instances or buckets, consid
* **Validation:** Add input validation rules to the `variables.tf` file to enfor
* **Dependency Management:** The code uses implicit dependencies (referencing ou

Example Usage:

```bash
terraform init
terraform plan
terraform apply
```

State Management:

* Terraform uses a state file to track the current infrastructure.

```
* This state file is typically stored locally, but can be managed remotely using
* The state file enables Terraform to perform efficient updates and deletions, u

**Important Considerations:**

* **Error Handling:** Terraform provides error handling mechanisms, which can be
* **Environment Variables:** You can use Terraform's `var.value` syntax to refer

**Reference Documentation:**

* Google Cloud Platform: https://cloud.google.com/
* Terraform: https://www.terraform.io/
* Terraform Google Provider: https://registry.terraform.io/providers/hashicorp/g
* Terraform Best Practices: https://cloud.google.com/docs/terraform/best-practic
```

The transformation from Bash to Terraform is highly organized and clear. The configuration is split into `variables.tf` and `main.tf`, enhancing readability and maintainability. `variables.tf` defines all input variables with descriptions and default values, ensuring flexibility. `main.tf` uses appropriate Terraform resources like `google_project_service`, `google_compute_instance`, and `google_storage_bucket`, properly configuring all services and infrastructure. Best practices such as descriptive naming, comments, and variable usage are applied, improving readability and reusability. The output also suggests optimizations like module use and input validation for better code quality and maintainability.

Summary

In this blog, we creatively transformed bash scripts into Terraform configurations using the Gemini 1.5 Flash LLM and Spring AI. By leveraging Retrieval-Augmented Generation (RAG), our solution ensures that the generated Terraform code not only replicates the functionality of the original bash script but also adheres to best practices for maintainability and scalability.

The key components of our solution include:

- **Ingestion Flow:** This involves collecting and processing Terraform best practices, converting them into embeddings, and storing them in the AlloyDB database.
- **Analysis Flow:** It includes utilizing semantic search to retrieve relevant best practices, constructing a prompt for the Gemini 1.5 Flash LLM, and generating well-structured Terraform code from the provided bash script.
- **Configuration and Implementation:** This step involves setting up necessary technologies including Java, Spring, Spring AI, Vertex AI, and AlloyDB to ensure seamless integration and efficient execution of the transformation process.

The benefits of this approach are manifold. It automates the labor-intensive task of converting bash scripts to Terraform, ensures adherence to best practices, and produces clear, maintainable, and scalable Terraform scripts. This significantly enhances the efficiency and reliability of managing GCP infrastructure.

References

RAG application is based on a collaboration between colleague Dan Dobrin and myself.

Gcp App Dev | Vertex AI | Vector Embeddings | Gemini | Google Cloud Platform

Published in Google Cloud - Community

74K followers · Last published 19 hours ago

Follow

A collection of technical articles and blogs published or curated by Google Cloud Developer Advocates. The views expressed are those of the authors and don't necessarily reflect those of Google.

Written by Yannipeng

8 followers · 6 following

Edit profile

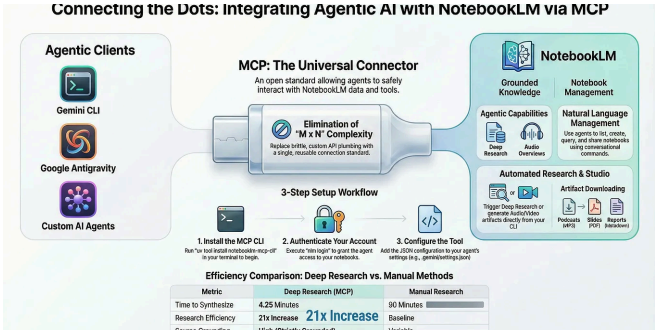
Developer at heart with java, k8, CI/CD, serverless, python, & genai chops. Currently a Google Cloud engineer specializing in application modernization.

No responses yet

Yannipeng

What are your thoughts?

More from Yannipeng and Google Cloud - Community



In Google Cloud - Commu... by Dazbo (Darren Les...

Integrate NotebookLM with Gemini CLI, Google Antigravity or Other...

Connect your AI agent to NotebookLM via MCP to autonomously research, summarize,...

Mar 1 890 1

In Google Cloud - Com... by Prashanth Subrahm...

Great-looking UIs with Google Stitch and Google Antigravity

First impressions matter. In modern applications, that impression is entirely...

Feb 23 142

In Google Cloud - Comm... by Rachael Deacon-s...

BigQuery Graph Series | Part 1: From “Dark Data” to Knowledge...

Feb 24 212 2

In Google Cloud - Community by The kube guy

Kubernetes v1.36 — Sneak Peek

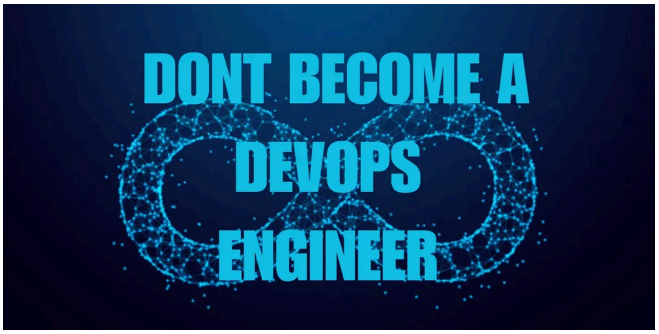
Kubernetes v1.36 is around the corner, and here’s what’s cooking behind the scenes

Mar 9 123 3

See all from Yannipeng

See all from Google Cloud - Community

Recommended from Medium

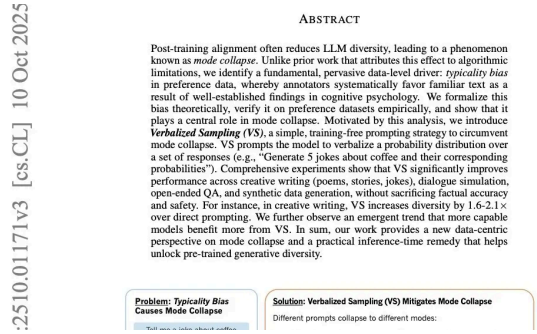


Dhanush N

Don’t Become a DevOps Engineer in 2026!

Stop writing YAML. Stop babysitting pipelines. The game has fundamentally...

Mar 2 233 3



In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

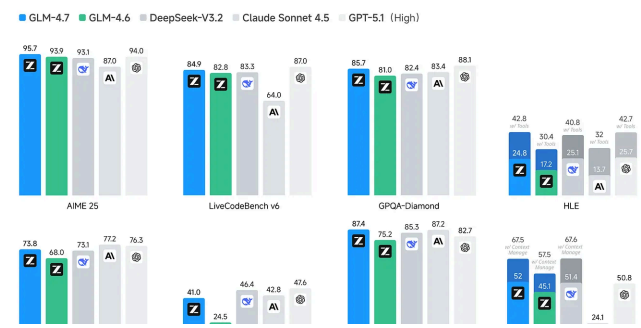
ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

Oct 19, 2025 25K 686

In AI Software Engineer by Joe Njenga

Anthropic Just Released Claude Code Course (And I Earned My...

Anthropic just launched their Claude Code in Action course, and I’ve just passed—how...



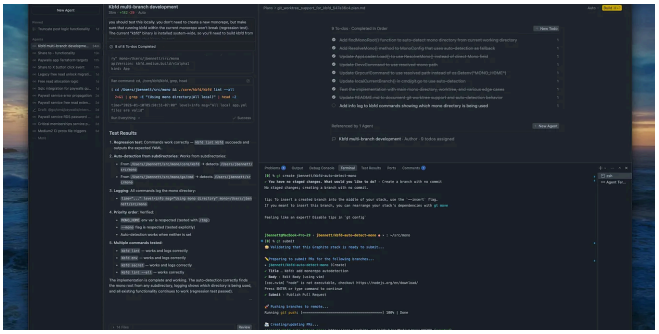
Agent Native

Local LLMs That Can Replace Claude Code

Editor’s note: While hardware tiers remain valuable context for this article, we’ve...

Jan 21 4K 64

Jan 20 1.8K 51



Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Jan 18 4.2K 103

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big...

Something strange happened in early 2026. Apple stores started running low on Mac...

Feb 15 5.6K 99

See more recommendations